

EVENT-DRIVEN ARCHITECTURE

Engineering Issues, Failures, Diagnostics & Resolutions

AWS Serverless Architecture with LocalStack

Author: Keh Essien

Email: emperordevops@gmail.com

GitHub: EmperorTechNation

A complete engineering runbook documenting architecture design, debugging methodology, AWS CLI workflows, failure scenarios, issue resolutions, deployment validation, and operational lessons learned while implementing a serverless event-driven architecture locally using LocalStack.

Project Overview

This document captures the real-world engineering issues, debugging processes, architecture decisions, CLI outputs, diagnostics, resolutions, and lessons learned while building an event-driven serverless architecture locally using AWS services on LocalStack. The architecture consists of AWS Lambda, Amazon SQS, Amazon DynamoDB, IAM Roles & Policies, CloudWatch Logs, LocalStack, and AWS CLI/awslocal.

Architecture Workflow

Producer Lambda → SQS Queue → Worker Lambda → DynamoDB

Core Services Used

- AWS Lambda — Serverless compute
 - Amazon SQS — Message queue for decoupling
 - DynamoDB — Persistent NoSQL storage
 - IAM — Permissions & execution roles
 - CloudWatch Logs — Logging & debugging
 - LocalStack — Local AWS cloud simulation
 - AWS CLI / awslocal — Infrastructure management
-

Issue 1 — Lambda Timeout Errors

Problem: Lambda invocations repeatedly timed out after 3 seconds.

Root Cause: Broken Lambda code and invalid AWS service connectivity configuration.

Diagnosis: Reviewed Lambda logs, isolated failing sections, and validated boto3 connectivity.

Resolution: Rebuilt Lambda logic and corrected endpoint configuration.

Result: Lambda returned successful responses with stable execution.

Issue 2 — LocalStack Networking Failure

Problem: Lambda containers could not communicate with LocalStack services.

Root Cause: Using localhost inside Docker-based Lambda containers.

Resolution: Replaced localhost with host.docker.internal:4566.

Result: Reliable SQS and DynamoDB communication established.

Issue 3 — SQS Event Source Mapping Failure

Problem: Messages entered SQS but Worker Lambda never executed.

Root Cause: Missing or disabled event source mapping.

Resolution: Created and enabled event source mapping using awslocal commands.

Verification: Confirmed State = Enabled.

Issue 4 — DynamoDB Insert Failures

Problem: Worker Lambda processed events but DynamoDB remained empty.

Root Cause: Incorrect DynamoDB endpoint configuration.

Resolution: Configured boto3 DynamoDB resource with host.docker.internal endpoint.

Result: Successful database persistence.

Issue 5 — Windows JSON Payload Problems

Problem: Lambda invocation failed because payload formatting was invalid.

Root Cause: Windows PowerShell encoding and AWS CLI handling issues.

Resolution: Used UTF-8 JSON payload files with fileb://payload.json.

Result: Reliable Lambda invocation workflow achieved.

Issue 6 — Lambda Pending State Errors

Problem: Lambda invocation attempted before deployment completed.

Root Cause: Function remained in Pending state.

Resolution: Waited until State = Active before invocation.

Result: Stable deployment workflow.

Issue 7 — Incorrect ZIP Packaging

Problem: Lambda deployments failed due to improper ZIP structure.

Resolution: Standardized packaging workflow using Compress-Archive from inside Lambda folders.

Result: Consistent deployments and reduced import errors.

CLI Errors & Solutions

ResourceConflictException → Wait until Lambda State = Active
InvalidRequestContentException → Use UTF-8 payloads with fileb://
EndpointConnectionError → Use host.docker.internal
AccessDeniedException → Attach correct IAM policies
QueueDoesNotExist → Verify Queue URL
ImportModuleError → Correct ZIP packaging

Failure Scenarios

Scenario A — Queue receives messages but Worker never executes.
Fix: Recreate event source mapping.

Scenario B — Lambda executes but DynamoDB remains empty.
Fix: Correct endpoint configuration.

Scenario C — Lambda cannot reach LocalStack services.
Fix: Replace localhost with host.docker.internal.

Scenario D — Lambda invocation fails immediately.
Fix: Use payload.json with fileb://.

Architecture Decisions

- Use LocalStack for local AWS simulation
 - Use awslocal only for consistency
 - Use SQS between Lambdas for decoupling
 - Use DynamoDB for lightweight persistence
 - Use fileb:// payloads for reliability
 - Use Python 3.11 runtime
 - Verify Active state before invocation
-

Lessons Learned

1. Distributed systems require strict validation.
 2. Networking is a major source of failure.
 3. Event-driven systems require observability.
 4. Deployment consistency matters.
 5. Structured runbooks improve troubleshooting efficiency.
-

Final Stable Architecture

Producer Lambda responsibilities:

- Accept payloads
- Validate input
- Send messages to SQS

Worker Lambda responsibilities:

- Consume SQS events
 - Parse queue messages
 - Write records into DynamoDB
-

Validation & Verification

- Lambda state verification — Passed
 - SQS queue validation — Passed
 - Event source mapping enabled — Passed
 - Producer Lambda invocation — Passed
 - Worker Lambda execution — Passed
 - DynamoDB insertion — Passed
 - End-to-end testing — Passed
-

Conclusion

This project successfully implemented a complete event-driven serverless architecture locally using LocalStack while documenting the real-world engineering issues, diagnostics, resolutions, and operational lessons learned throughout deployment and operations. Final Architecture:

Producer Lambda → SQS → Worker Lambda → DynamoDB
